

05

EDA

Exploratory Data Analysis

Getting to know your data before you build anything

AI/ML Engineer Learning Series · Deep Dive Edition

EDA means	Looking at your data carefully to understand it
Tools used	Pandas + Matplotlib + Seaborn together
Goal	Find patterns, problems, and surprises in the data
When	Right after getting data, BEFORE building a model
Why it matters	A model is only as good as the data you feed it

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is EDA?	The idea and why it matters
2	The EDA Checklist	A step-by-step plan you can reuse
3	First Look	Shape, head, info, dtypes
4	Summary Statistics	describe() and what the numbers mean
5	Missing Values	Find them and decide what to do
6	One Variable (Univariate)	Histograms, value counts, distributions
7	Two Variables (Bivariate)	Scatter, groupby, comparisons
8	Outliers	Spotting strange values
9	Correlation	Which features move together
10	Full EDA Walkthrough	A real start-to-finish example
11	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference of every command

1. What is EDA?

EDA stands for **Exploratory Data Analysis** (■■■■ ■■■■■■■■ ■■■■■■■■ — ■■■■ ■■■■■ ■■■■■). It simply means: before you do anything fancy, you sit with your data and get to know it. You look at it from many angles, draw charts, and ask questions.

Think of it like a doctor checking a patient before giving medicine. The doctor first looks, asks questions, and runs tests. Only then do they decide treatment. EDA is you checking your data before you decide which model to build.

During EDA you are trying to answer questions like:

- How big is the data? How many rows and columns?
- Are there missing (■■■■■) values? How many?
- What does each column look like — small numbers, big, text?
- Are there strange values (outliers — ■■■■■■■■■■ ■■■■)?
- Do any columns move together (correlation — ■■■■■■ ■■■■■■-■■■■)?
- Is the data balanced, or is one group much bigger?

■ *EDA is the most skipped but most important step. People rush to build models. But good engineers spend a lot of time here — because understanding the data saves you later.*

2. The EDA Checklist

Here is a simple plan you can follow every single time you get a new dataset. Do these steps in order:

Step	What you do	Main tool
1. Shape	Check rows and columns	df.shape
2. Peek	Look at the first few rows	df.head()
3. Types	Check column types	df.info()
4. Stats	Get summary numbers	df.describe()
5. Missing	Count empty values	df.isnull().sum()
6. One column	Study each column alone	histplot, value_counts
7. Two columns	Compare pairs of columns	scatterplot, groupby
8. Outliers	Find strange values	boxplot
9. Correlation	See what moves together	heatmap
10. Notes	Write down what you found	your brain!

3. First Look at the Data

The very first thing: load the data and look at its size and shape. These commands take 2 seconds and tell you a lot.

```
import pandas as pd
df = pd.read_csv('titanic.csv')

df.shape      # (891, 12) → 891 rows, 12 columns
df.head()     # first 5 rows – a quick peek
df.tail()     # last 5 rows
df.columns    # list of all column names
df.info()     # column types + how many non-empty values
df.dtypes     # just the type of each column
```

df.info() is a favourite. In one view it shows every column, its type, and how many values are NOT empty. If a column has fewer non-empty values than the total row count, you know it has missing data.

4. Summary Statistics

df.describe() gives you key numbers for every number column at once. Here is what each one means in plain words:

Stat	Plain meaning
count	How many values are there (not empty)
mean	The average (■■■■)
std	Standard deviation — how spread out the values are
min	The smallest value
25%	One quarter of values are below this
50%	The middle value (median — ■■■■■■)
75%	Three quarters of values are below this
max	The biggest value

```
df.describe()      # stats for number columns
df.describe(include='object') # stats for text columns
df.describe(include='all')   # everything
```

■ *Quick trick: if mean and 50% (median) are very different, the data is skewed (■■■■■■■ ■■■■ ■■■■) — often because of a few very big or very small values.*

5. Missing Values

Real data almost always has holes — values that are missing (■■■■). You must find them and decide what to do. Ignoring them causes errors later.

```
df.isnull().sum()           # count missing per column
df.isnull().sum() / len(df) * 100  # percentage missing
df.isnull().sum().sort_values(ascending=False)  # worst first
```

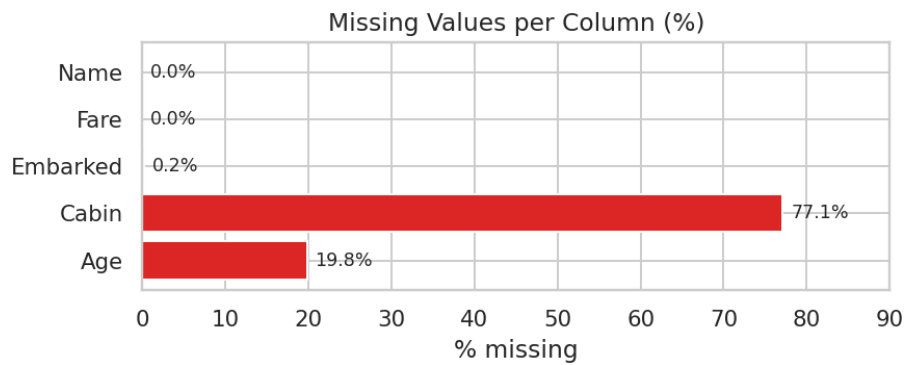


Fig 1 — A bar chart of missing values per column. Cabin is mostly empty (77%).

Once you know what's missing, you choose an action for each column:

Situation	What to do
Only a few missing	Fill with mean / median / most common value
Most of the column missing	Drop the whole column (it's not useful)
A few rows missing in key column	Drop just those rows
Missing has a meaning	Make a new category like 'Unknown'

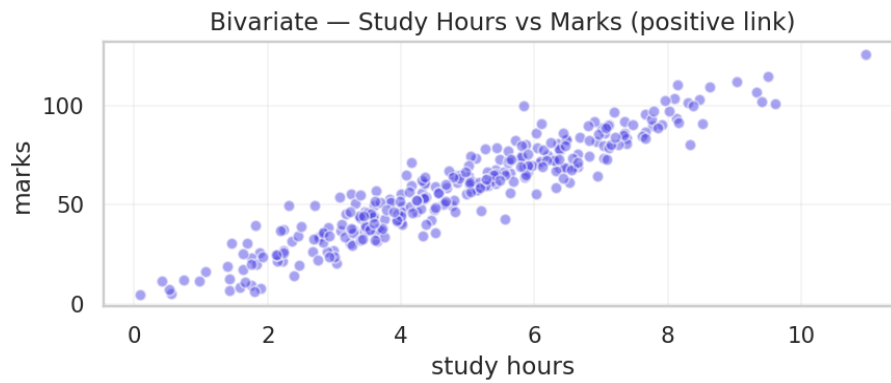


Fig 3 — Study hours vs marks. Dots rising together = a positive relationship.

```
# Number vs Number → scatter plot
sns.scatterplot(data=df, x='study_hours', y='marks')

# Category vs Number → boxplot or grouped average
sns.boxplot(data=df, x='Pclass', y='Fare')
df.groupby('Pclass')['Fare'].mean()

# Category vs Category → counts split by a second column
sns.countplot(data=df, x='Sex', hue='Survived')
pd.crosstab(df['Sex'], df['Survived'])    # a count table
```

8. Outliers — Strange Values

An **outlier** (■■■■■■■■■■) is a value that is very different from the rest — much bigger or much smaller. Example: in a class where everyone is 20-25 years old, a person listed as 200 years old is an outlier (probably a typo). Outliers can confuse your model, so you need to find them.

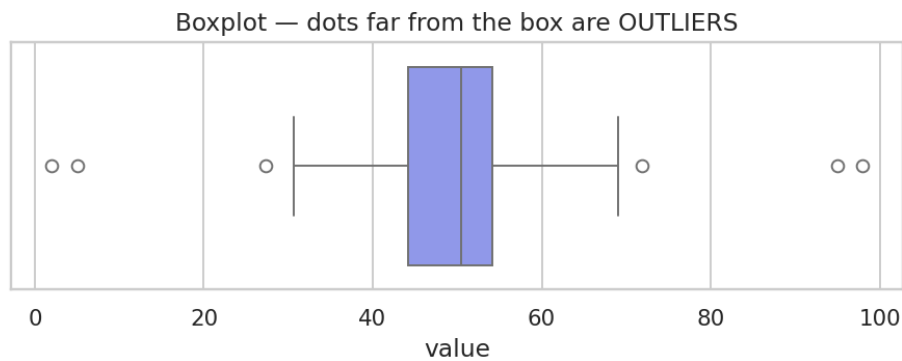


Fig 4 — A boxplot. The dots sitting far away from the box are outliers.

```
# Boxplot is the fastest way to SEE outliers
sns.boxplot(x=df['Age'])

# Find them with numbers (IQR method)
Q1 = df['Age'].quantile(0.25) # 25% point
Q3 = df['Age'].quantile(0.75) # 75% point
IQR = Q3 - Q1                # middle range

low  = Q1 - 1.5 * IQR
high = Q3 + 1.5 * IQR
outliers = df[(df['Age'] < low) | (df['Age'] > high)]
print(outliers)
```

■ Not every outlier is bad! Sometimes the strange value is real and important (like a fraud transaction). Look first, then decide: keep it, fix it, or remove it.

9. Correlation — What Moves Together

Correlation (■■■■■■■■■■) measures how strongly two number columns move together. It is a number between -1 and +1:

- **+1** = perfect positive: when one goes up, the other goes up too.
- **0** = no link: they don't affect each other.
- **-1** = perfect negative: when one goes up, the other goes down.

A heatmap shows all correlations at once with color. This helps you pick useful features for your model and spot columns that say the same thing.

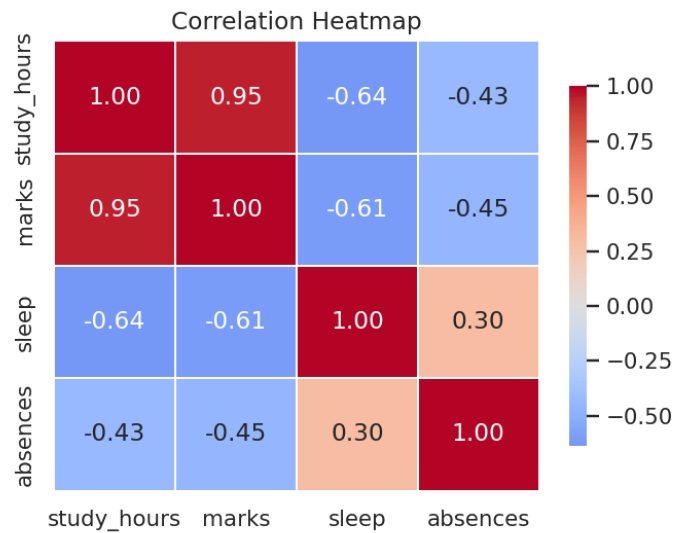


Fig 5 — Correlation heatmap. Study hours and marks are strongly linked (red).

```
# Table of correlations between all number columns
df.corr(numeric_only=True)

# Draw it as a heatmap
sns.heatmap(df.corr(numeric_only=True),
            annot=True, cmap='coolwarm', center=0, fmt='.2f')
```

■ Important: correlation is NOT the same as cause. Ice cream sales and drowning both go up in summer — but ice cream doesn't cause drowning. The hot weather causes both.

10. A Full EDA Walkthrough

Here is a complete EDA from start to finish on the famous Titanic dataset. This is exactly the kind of code you write at the start of a real project.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style='whitegrid')

# --- Step 1-3: First look ---
df = pd.read_csv('titanic.csv')
print(df.shape)          # (891, 12)
print(df.head())
print(df.info())

# --- Step 4: Summary stats ---
print(df.describe())

# --- Step 5: Missing values ---
print(df.isnull().sum().sort_values(ascending=False))
# Cabin 687, Age 177, Embarked 2

# --- Step 6: One column ---
sns.histplot(data=df, x='Age', bins=30, kde=True); plt.show()
print(df['Sex'].value_counts())

# --- Step 7: Two columns (key question: who survived?) ---
sns.countplot(data=df, x='Sex', hue='Survived'); plt.show()
print(df.groupby('Sex')['Survived'].mean())
# women survived far more than men

# --- Step 8: Outliers ---
sns.boxplot(x=df['Fare']); plt.show() # a few very high fares

# --- Step 9: Correlation ---
sns.heatmap(df.corr(numeric_only=True), annot=True,
            cmap='coolwarm'); plt.show()
```

What we learned from this EDA: the data has 891 rows; Cabin is mostly empty so we'll drop it; Age has some gaps to fill; women survived much more than men; and a few fares are unusually high. Now we know our data — and we're ready to build a model.

11. Common Mistakes

Mistake	Why it's a problem	Fix
Skipping EDA	You miss problems in the data	Always explore first

Ignoring missing values	Causes errors and bad models	Check and handle them
Deleting all outliers	You may delete real, useful data	Look before removing
Confusing correlation/cause	Wrong conclusions	Correlation is not cause
Only looking at numbers	Miss the shape of data	Draw charts too
Not writing down findings	You forget what you learned	Keep short notes

Quick Reference Cheatsheet

Task	Code
Shape	<code>df.shape</code>
First rows	<code>df.head()</code>
Column types	<code>df.info()</code>
Summary stats	<code>df.describe()</code>
Missing count	<code>df.isnull().sum()</code>
Missing percent	<code>df.isnull().sum() / len(df) * 100</code>
Unique values	<code>df['col'].nunique()</code>
Count categories	<code>df['col'].value_counts()</code>
Histogram	<code>sns.histplot(data=df, x='col', kde=True)</code>
Count bar	<code>sns.countplot(data=df, x='col')</code>
Scatter	<code>sns.scatterplot(data=df, x='a', y='b')</code>
Boxplot	<code>sns.boxplot(data=df, x='a', y='b')</code>
Group average	<code>df.groupby('a')['b'].mean()</code>
Cross table	<code>pd.crosstab(df['a'], df['b'])</code>
Skew	<code>df['col'].skew()</code>
Quantile	<code>df['col'].quantile(0.25)</code>
Correlation table	<code>df.corr(numeric_only=True)</code>
Heatmap	<code>sns.heatmap(df.corr(), annot=True)</code>

Next Topic: Scikit-learn — the main toolbox for building classical machine learning models. Now that your data is clean and understood, it's time to teach the computer.